

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

**Факультет компьютерных наук
Департамент программной инженерии**

Пугач Виктория Павловна, группа МСПИН251

R1 Техническое задание
на курсовую работу

Архитектура образовательной веб-платформы для совместного
программирования с поддержкой CRDT и генерацией учебных подсказок
**(Design of a Web-Based Educational Platform for Collaborative
Programming with CRDT and Hint Generation)**

Руководитель:
доцент департамента программной инженерии,
Брейман Александр Давидович

11/12/2025

Москва, 2025 г.

Оглавление

1 Введение.....	3
1.1 Предметная область	3
1.2 Актуальность	3
1.3 Проблема.....	4
1.3.1 Суть проблемы	4
1.3.2 Существующие решения	4
1.3.3 Преимущества предлагаемой системы	5
1.4 Пользователи и заинтересованные стороны	6
1.5 Цель и задачи	6
1.5.1 Цель разработки	6
1.5.2 Задачи, решаемые системой:	6
1.5.3 Ожидаемые результаты	7
2 Входные и выходные данные.....	7
3 Пользовательские сценарии (Use Cases).....	7
4 Требования к системе	10
4.1 Функциональные требования	10
4.2 Нефункциональные требования	11
4.3 Критерии оценки решения	12
5 Выводы	14
6 Список источников	15

1 Введение

Данный документ представляет собой техническое задание (ТЗ) на проектирование прототипа веб-платформы в сфере образовательных технологий (EdTech). Платформа предназначена для организации совместного обучения детей программированию в режиме реального времени.

1.1 Предметная область

Предметная область относится к сфере EdTech и фокусируется на создании распределенных коллаборативных сред, адаптированных для детской аудитории. Контекст задачи включает интеграцию алгоритмов согласованности распределенных данных (CRDT) [1] для бесконфликтного редактирования и системы интеллектуальной генерации адаптивных учебных подсказок. Проектирование системы будет вестись с применением принципов предметно-ориентированного проектирования (DDD) [2, 3] для управления сложностью и обеспечения четкого разделения ответственности между компонентами.

1.2 Актуальность

Актуальность и значимость темы обусловлены растущим спросом на интерактивные и социальные формы онлайн-обучения. Существующие инструменты либо не обеспечивают технически надежной коллаборации, либо не предлагают педагогически обоснованной поддержки, адаптированной для детского восприятия [4, 5]. Практическая значимость подкреплена сотрудничеством с образовательной организацией «Инжиниум», что обеспечивает связь с реальными потребностями пользователей.

Особую актуальность проекту придает текущая ситуация с доступностью образовательных IT-инструментов. В последние годы ряд популярных зарубежных платформ для обучения детей программированию и цифровому творчеству (таких как Roblox, Tinkercad, частично Delightex) либо

прекратили работу на российском рынке, либо оказались под санкционными ограничениями. Например, массовая блокировка Roblox, который активно использовался на курсах по геймдеву и основам программирования, наглядно продемонстрировала риски зависимости от иностранных сервисов. Это создало острую проблему платформозависимости для российских EdTech-проектов и образовательных учреждений. Таким образом, разработка собственной, технологически независимой и педагогически эффективной платформы становится не только методической, но и стратегической задачей, направленной на обеспечение устойчивости образовательного процесса.

1.3 Проблема

1.3.1 Суть проблемы

Ключевая проблема заключается в отсутствии единой цифровой среды, которая одновременно решает технологические задачи надежной совместной работы и педагогические задачи адаптивной поддержки ученика в процессе обучения программированию. Это приводит к ситуации, когда для проведения одного группового занятия преподаватель и ученики вынуждены использовать несколько разнородных инструментов (отдельный редактор кода, среду для видеосвязи, платформу для обмена файлами и ручной проверки), ни один из которых не обеспечивает целостного и комфортного образовательного опыта.

1.3.2 Существующие решения

С точки зрения технологии **коллаборации**:

- общие онлайн-редакторы (на базе решений для документов) не поддерживают синтаксис программирования, авто-дополнение и структурированную работу с кодом;
- профессиональные IDE с плагинами для совместной работы часто требуют сложной настройки, создают высокую нагрузку на сеть и не гарантируют согласованности данных при одновременном редактировании, приводя к конфликтам и потере введенного кода;
- большинство решений имеют высокую латентность, что разрушает ощущение «живого» совместного творчества.

С точки зрения **педагогики** и поддержки ученика:

- стандартные компиляторы и интерпретаторы выводят сообщения об ошибках на сложном техническом языке, непонятном для детей, вызывая фрустрацию и демотивацию;
- образовательные платформы с автоматической проверкой часто ограничиваются бинарным верно/неверно, не объясняя причину ошибки и не предлагая шаги для ее исправления, что критически важно для формирования навыка отладки;
- отсутствует персонализация обратной связи: система не учитывает индивидуальный прогресс ученика, его предыдущие ошибки и когнитивные особенности.

С точки зрения экосистемы и **доступности**:

- как отмечено во введении, рынок столкнулся с проблемой платформозависимости. Популярны зарубежные сервисы, которые ранее использовались в обучении, стали недоступны, создав вакуум и подчеркнув необходимость в собственных, контролируемых решениях.

1.3.3 Преимущества предлагаемой системы

Для ученика: формирование позитивного опыта совместной работы и развитие навыка самостоятельного поиска и исправления ошибок через понятные подсказки.

Для преподавателя: сокращение времени на техническую поддержку и рутинные объяснения однотипных ошибок.

Для образовательной организации («Инжиниум»): Получение независимого, надежного и методически адекватного инструмента, снижающего риски сбоев учебного процесса из-за внешних факторов. Возможность накопления и анализа данных об учебной активности для совершенствования образовательных программ.

При помощи системы получится избежать:

- снижения мотивации и вовлеченности учеников из-за фрустрации от технических сбоев или непонятных сообщений об ошибках;
- низкую эффективность групповых заданий онлайн, ведущую к потере ключевого преимущества обучения — социализации и коллективного решения задач;
- повышенную нагрузку на преподавателя, вынужденного решать конфликты при решении задач по совместной разработке вручную, а также выступать единственным источником подсказок.

1.4 Пользователи и заинтересованные стороны

Роль	Уровень доступа	Основные потребности
Ученик (ребенок 6-17 лет)	Основной пользователь. Доступ к своим проектам и сессиям, получение подсказок.	1. Простой, яркий и интуитивный интерфейс. 2. Возможность видеть изменения других в реальном времени без «глюков». 3. Получение ясной, дружелюбной помощи при ошибке, а не сложного технического текста. 4. Элементы игры, похвалы и визуального прогресса.
Преподаватель	Расширенный доступ. Создание/управление сессиями, просмотр общей активности.	1. Легко запустить групповую работу и пригласить учеников. 2. Видеть общую картину: кто что делает, какие ошибки возникают у группы. 3. Возможность дать персональный совет конкретному ученику.
Администратор (заказчик «Инжиниум»)	Полный доступ к админ-панели и аналитике.	1. Надежность и стабильность платформы во время занятий. 2. Возможность собирать анонимизированные метрики об использовании для улучшения методик. 3. Техническая возможность масштабирования системы.

Таблица 1 – Основные заинтересованные стороны разрабатываемого решения

1.5 Цель и задачи

1.5.1 Цель разработки

Спроектировать и обосновать архитектуру веб-платформы, которая технически интегрирует механизм совместного редактирования на основе CRDT с педагогической системой генерации адаптивных подсказок для детей.

1.5.2 Задачи, решаемые системой

- 1) обеспечить бесконфликтную синхронизацию кода между множеством пользователей в реальном времени;
- 2) автоматически обнаруживать типовые ошибки в коде и генерировать для них пояснения на языке, понятном ребенку;
- 3) предоставить веб-интерфейс, соответствующий принципам UI/UX для детей;
- 4) гарантировать низкую задержку для ощущения «живого» взаимодействия.

1.5.3 Ожидаемые результаты

- 1) Документированная архитектура системы, утвержденная научным руководителем и представителем заказчика.
- 2) Функционирующий прототип, демонстрирующий:
 - a. модуль совместного редактирования с базовой реализацией CRDT;
 - b. сервис анализа кода и генерации простых подсказок (на 3-5 простых типов ошибок);
 - c. веб-интерфейс для сессии.
- 3) Технические метрики прототипа (KPI):
 - a. задержка синхронизации (от действия до отражения у других): < 100 мс при 10 пользователях;
 - b. время отклика на генерацию подсказки: < 2 секунд.
- 4) Экспертная оценка: релевантность и понятность генерируемых подсказок признана удовлетворительной педагогом-экспертом от «Инжиниум».

1.6 Входные и выходные данные

Категория	Описание	Формат / Источник
Входные данные	1. Операции редактирования: ввод символа, удаление, перенос строки.	Поток операций в формате CRDT. Источник: действия пользователя в веб-редакторе. Транспорт: WebSocket.
	2. Исходный код для анализа: Снимок кода или контекст ошибки.	Текстовые данные (фрагмент кода на Python/JS/Lua etc.). Источник: редактор. Транспорт: HTTP-запрос к API анализа.
	3. Команды управления сессией: создание, присоединение, запрос подсказки.	Структурированные JSON-команды через WebSocket/HTTP API.
Выходные данные	1. Синхронизированное состояние: Актуальная версия кода для всех клиентов.	Рассылка CRDT-операций или патчей состояния через WebSocket.
	2. Учебные подсказки: Текстовые и визуальные рекомендации.	JSON-объект: { "type": "hint", "message": "Текст...", "location": { "line": X }, "visualCue": "highlight" }.
	3. Системная информация: Список онлайн-пользователей, статус подключения.	Данные для отображения в UI-виджетах интерфейса.

Таблица 2 – Входные и выходные данные системы

2 Пользовательские сценарии (Use Cases)

Название	Совместное редактирование кода в сессии
Рамки применения	Система совместного программирования
Значимость	Ключевая задача
Первичный актер	Ученик (ребенок 6-17 лет)
Вторичные актеры	Сервер синхронизации (обрабатывает CRDT-операции, обеспечивает рассылку)
Базовый поток	А. Ученик редактирует код. В. Система отправляет CRDT-операцию на сервер. С. Сервер рассылает операцию всем участникам. Д. Система каждого ученика применяет операцию. Е. Код у всех отображается идентично.
Альтернативные потоки	A1. Восстановление при разрыве связи: A1-A. Потеря соединения → буферизация изменений. A1-B. Восстановление → отправка операций. A1-C. CRDT обеспечивает согласованность. D1. Конфликт операций: Автоматическое разрешение CRDT. D2. Длительный разрыв (>5 мин): Предложение переподключиться.
Предусловия	Ученик авторизован и находится в активной сессии совместной работы.
Постусловия	Код у всех участников сессии синхронизирован.

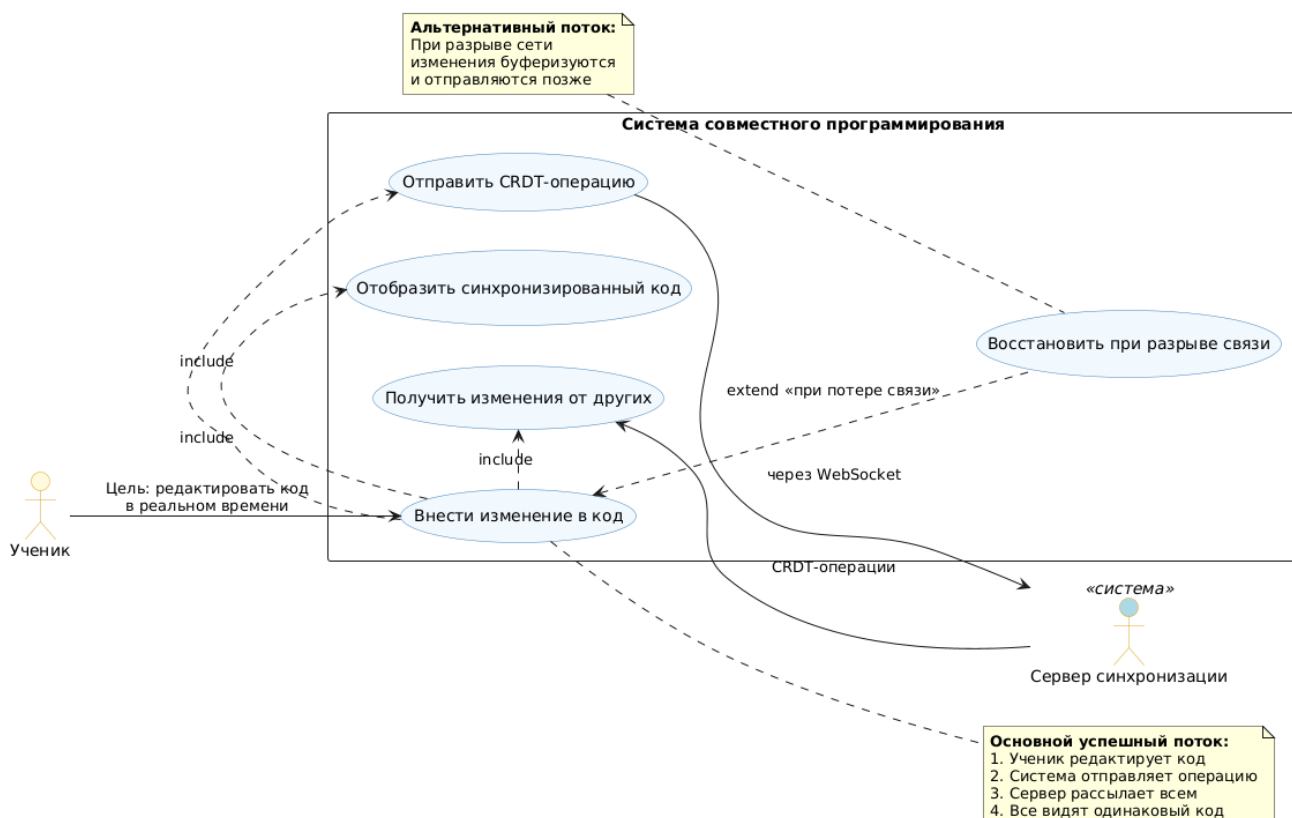


Рисунок 1 – Use Case: Совместное редактирование кода в сессии

Название	Получение подсказки при ошибке
Рамки применения	Система интеллектуальной поддержки обучения
Значимость	Ключевая задача (педагогическая составляющая)
Первичный актер	Ученик / Система
Вторичные актеры	Сервис анализа кода (статический анализ, AST, генерация подсказок)
Базовый поток	A. Система обнаруживает ошибку в коде. B. Код отправляется в сервис анализа. C. Сервис генерирует адаптивную подсказку. D. Подсказка отображается с подсветкой места ошибки. E. Ученик получает понятное объяснение.
Альтернативные потоки	A1. Ручной запрос: Ученик нажимает "Помощь" → анализ → подсказка по логике задачи. C1. Сервис недоступен: Сообщение "Сервис подсказок временно недоступен". C2. Неопределенная ошибка: Общая подсказка "Проверь символы в строке". D1. Частые запросы: Временное ограничение с советом подумать самостоятельно.
Предусловия	Ученик работает в редакторе. Для автоматического потока — есть ошибка в коде.
Постусловия	Ученик получает адаптированную подсказку. Ошибка становится понятной/исправляется.

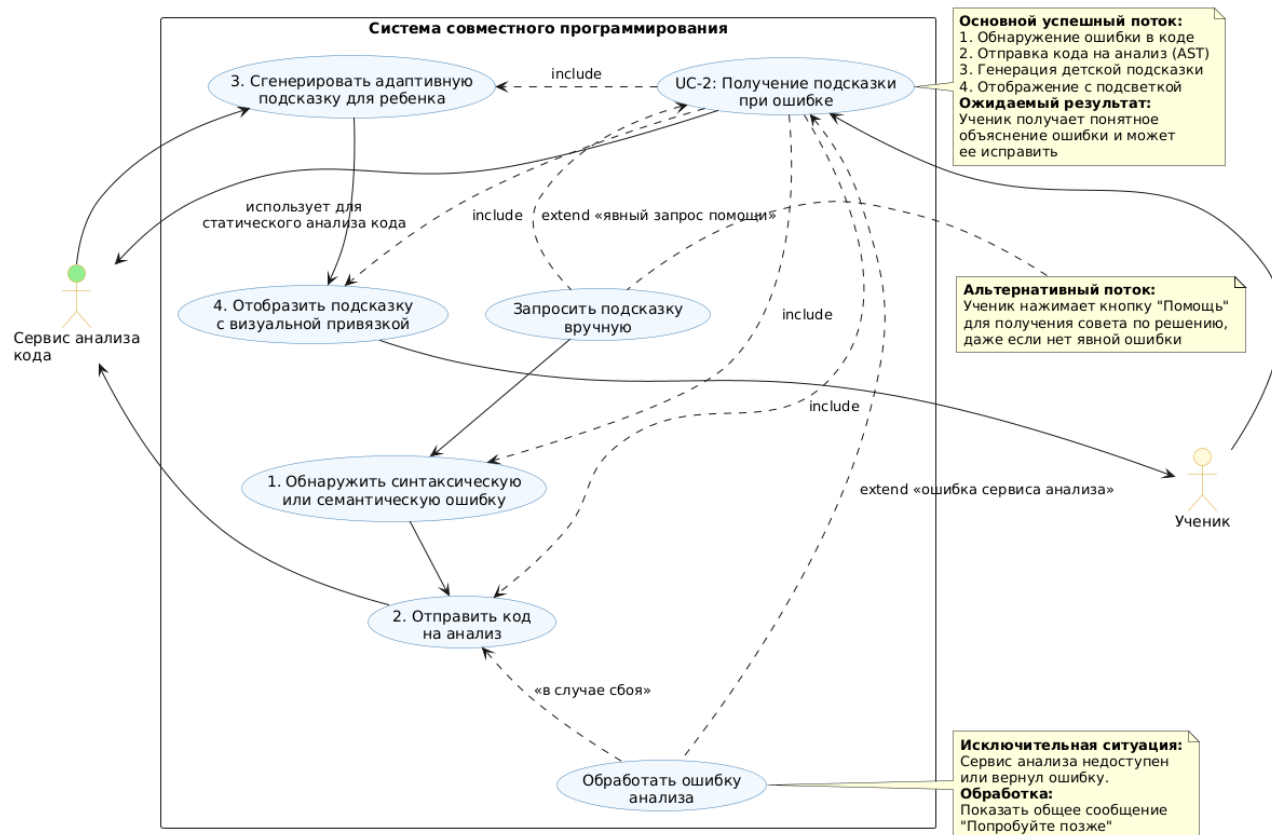


Рисунок 2 – Use Case: Получение подсказки при ошибке

3 Требования к системе

3.1 Функциональные требования

Ядро совместной работы:

1. Система должна предоставлять веб-интерфейс с текстовым редактором кода, поддерживающим подсветку синтаксиса для языков, используемых в обучении детей (например, Python, JavaScript).
2. Система должна позволять пользователю создавать сессию совместной работы и предоставлять другим пользователям уникальную ссылку для присоединения.
3. Система должна реализовывать механизм синхронизации изменений кода в реальном времени для всех участников сессии с использованием алгоритмов CRDT для гарантированной бесконфликтной конвергенции данных.
4. Система должна отображать список активных пользователей в текущей сессии с индикацией их активности (курсор, выделение текста).
5. Система должна сохранять историю изменений (snapshot) кода в сессии для возможности отката.

Интеллектуальная поддержка и подсказки:

6. Система должна в фоновом режиме выполнять статический анализ набираемого кода, строя Абстрактное Синтаксическое Дерево (AST).
7. Система должна автоматически обнаруживать набор типовых синтаксических и семантических ошибок, характерных для начинающих программистов (несоответствие скобок, отсутствие объявления переменной, синтаксические ошибки).
8. Для обнаруженной ошибки система должна генерировать учебную подсказку, текст которой соответствует принципам адаптации для детского восприятия: краткость, конкретность, использование знакомых понятий, дружелюбный тон, избегание сложной терминологии.

9. Подсказка должна быть визуально привязана к месту ошибки в редакторе (например, через подсветку, подчеркивание, всплывающую плашку).

10. Система должна предоставлять пользователю (ученику) возможность явно запросить подсказку через интерфейс, если он испытывает затруднения, не связанные с синтаксической ошибкой.

11. Преподаватель должен иметь возможность вручную добавлять текстовые комментарии или подсказки в код для всех участников сессии.

Управление и интерфейс:

12. Система должна предоставлять базовую аутентификацию и регистрацию пользователей.

13. Система должна иметь раздел «Мои проекты/сессии» для доступа к созданным и активным сессиям.

14. Интерфейс должен включать элементы геймификации и позитивного подкрепления: визуальные значки за исправление ошибок, прогресс-бары, похвальные сообщения.

15. Система должна предоставлять RESTful API для интеграции сервиса анализа кода и генерации подсказок с другими системами (в перспективе).

3.2 Нефункциональные требования

1. Производительность. Задержка синхронизации изменений кода (от действия одного пользователя до отображения у другого) должна составлять менее 100 миллисекунд при нагрузке в 10 одновременно активных пользователей в одной сессии.
2. Масштабируемость. Архитектура сервиса реального времени (WebSocket + CRDT) должна быть спроектирована с

возможностью горизонтального масштабирования для поддержки роста количества параллельных сессий.

3. Надежность. Клиентское приложение должно обеспечивать офлайн-устойчивость — сохранять локальную копию неотправленных изменений и автоматически восстанавливать состояние сессии после разрыва соединения. Алгоритм CRDT должен гарантировать конечную согласованность данных.
4. Удобство использования. Веб-интерфейс должен быть интуитивно понятен для детей 6-17 лет. Оценка по упрощенному юзабилити-тесту с представителями аудитории должна быть не ниже 70 баллов.
5. Безопасность. Все данные должны передаваться по защищенным протоколам (HTTPS/WSS). Доступ к сессии должен контролироваться. Пользовательские данные должны шифроваться при хранении.
6. Интегрируемость. Архитектура должна быть модульной, четко разделяя компоненты совместного редактирования (CRDT-сервис), анализа кода (сервис подсказок) и бизнес-логики (управление пользователями, сессиями) для упрощения дальнейшего развития (например, интеграции AI-моделей).
7. Архитектурная четкость. Архитектура системы должна быть спроектирована с выделением ограниченных контекстов в соответствии с принципами DDD для разделения доменов совместного редактирования, педагогического анализа и управления обучением.

3.3 Критерии оценки решения

Система считается соответствующей требованиям при одновременном выполнении критериев по трём направлениям: функциональная полнота, техническая эффективность и педагогическая адекватность.

Категория	Ключевые критерии и целевые показатели	Методы проверки
Функциональные	• Реализованы все 15 функциональных требований из раздела 3.1. • Система обеспечивает	1. Ручное тестирование по чек-листу сценариев. 2. Приемочное

	бесконфликтное совместное редактирование.	тестирование с заказчиком.
Технические	• Задержка синхронизации: < 100 мс при 10 пользователях. • Время генерации подсказки: < 2 секунд. • Надежность: Восстановление после разрыва связи без потери данных.	1. Нагрузочное тестирование. 2. Тестирование отказоустойчивости.
Пользовательские	• Удобство интерфейса для детей: Оценка ≥ 70 баллов по шкале SUS. • Понятность подсказок: Экспертная оценка педагогом $\geq 80\%$. • Общая удовлетворенность: Средняя оценка ≥ 4.0 из 5.	1. Юзабилити-тестирование с фокус-группой детей. 2. Экспертная оценка методистом «Инжиниум». 3. Пилотное занятие с обратной связью.

Порядок и условия успеха:

1. Выполнение всех функциональных требований.
2. Подтверждение технических показателей тестами.
3. Получение положительной обратной связи от конечных пользователей (детей) и экспертов (педагогов).

4 Выводы

Проведенный анализ предметной области и требований образовательной организации «Инжиниум» позволил сформулировать детальное техническое задание на проектирование прототипа образовательной веб-платформы. Ключевой вывод заключается в том, что для решения выявленной проблемы необходим синтез технологий распределенных систем и педагогически ориентированной разработки в рамках единой архитектуры.

Основные положения, которые легли в основу ТЗ:

1. Проблема носит комплексный (технико-педагогический) характер, что отражено в структуре требований.
2. Целевая аудитория — дети — предъявляет особые требования к производительности (<100 мс), надежности (офлайн-устойчивость) и, главное, к дизайну интерфейса и формулировок обратной связи.
3. Сотрудничество с заказчиком («Инжиниум») обеспечивает практическую ориентированность требований и валидность критериев оценки, особенно в части педагогической эффективности подсказок.

Открытые вопросы для дальнейшей проработки:

1. Технический выбор: Определение конкретной библиотеки/реализации CRDT для стека JavaScript/Node.js и инструмента для статического анализа (например, на основе Babel Parser, Tree-sitter).
2. Педагогическая детализация: Формализация полного набора правил и шаблонов для генерации подсказок под конкретные языки и типы ошибок, используемые в учебных программах «Инжиниум».
3. Дизайн и геймификация: Разработка детальных UI/UX-макетов, воплощающих принципы визуальных метафор, интерактивности и позитивного подкрепления, заявленные в мотивации проекта.

5 Список источников

- 1) M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," *Lecture Notes in Computer Science*, pp. 386–400, 2011, doi: https://doi.org/10.1007/978-3-642-24550-3_29
- 2) V. Vernon, "Domain-Driven Design Distilled PDF Scan to Download." Accessed: Nov. 11, 2025. [Online]. Available: <https://cdn.bookey.app/files/pdf/book/en/domain-driven-design-distilled.pdf>
- 3) Э. Эванс, Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем. Москва: Вильямс, 2011
- 4) T. Delev and D. Gjorgjevikj, "Static analysis of source code written by novice programmers," 2017 IEEE Global Engineering Education Conference (EDUCON), Athens, Greece, 2017, pp. 825-830, doi: 10.1109/EDUCON.2017.7942942
- 5) T. Crow, A. Luxton-Reilly, and B. Wuensche, "Intelligent tutoring systems for programming education," *Proceedings of the 20th Australasian Computing Education Conference on - ACE '18*, 2018, doi: <https://doi.org/10.1145/3160489.3160492>